
The Case For WAL-Logging `pg_upgrade`

Bohyun Lee (patch [GitHub repo](#))

Problem of stock `pg_upgrade`. Current `pg_upgrade` writes the files of the new cluster directly, emitting no WAL for the changes it makes. This creates two fundamental limitations. The first is a **replication gap**. Since the upgrade never enters the WAL stream, standbys cannot adopt it through replication and must be taken offline and resynchronized from the primary via `rsync` [1]. The second is a **durability gap** in the backup chain. The upgraded cluster begins with a new WAL history that is disconnected from the history of the old cluster. Consequently, if the upgraded primary fails before a new base backup is taken, transactions committed after the upgrade cannot be recovered. This window may remain open for a long time, if the database is large and taking a backup requires a long time.

Closing replication and durability gaps. The patch closes both gaps by atomically WAL-logging the after-images generated by `pg_upgrade` upon successful completion. The upgrade becomes part of the WAL stream and can be propagated to standbys through standard streaming replication, eliminating the need for offline `rsync`-based resynchronization. Furthermore, because the upgraded cluster continues the existing WAL history rather than creating a new one, the upgrade and all subsequent modifications remain recoverable through WAL replay, preserving durability even before a new base backup is taken.

Workflow with primary/standby with the proposed interface. (1) `pg_upgrade --wal-upgrade-signal-handoff` triggers the primary to shutdown and standbys to pause. (2) `pg_upgrade --wal-upgrade` performs the upgrade (3) and *captures it as WAL*. (4) The primary goes live on restart with the new version. (5) Each standby comes up on the new version over its retained data directory, streams and replays the upgrade WALs (6) and becomes a hot standby once the WALs are fully replayed.

Recovery workflow with primary/backup with the proposed interface. (1) Restore the last pre-upgrade base backup. (2) PITR-replay archived WAL up to the upgrade point. (3) Restart on the new version and replay the upgrade WAL. (4) Keep replaying the new-version WAL to recover the post-upgrade transactions. (5) Promote, fully recovered without new base backup.

Expected benefits. Upgrading a replicated cluster reduces to upgrading the primary with minimal cluster coordination. Standbys catch up by themselves over their existing replication link, without offline `rsync` or a rebuild. It also guarantees durability of post-upgrade modifications even without a new base backup. Such benefits suggest that perhaps `--wal-upgrade` should even be the *default behavior*.

1. Proposal Overview

2. [Proposed Interface](#)
3. [Primary-standby Workflow with the New Interface](#)
4. [How --wal-upgrade Enables Continuous Archiving](#)
5. [Added TAP Tests](#)
6. [Example Use Case: k8s Zalando \(Spilo\)](#)

2. Proposed Interface

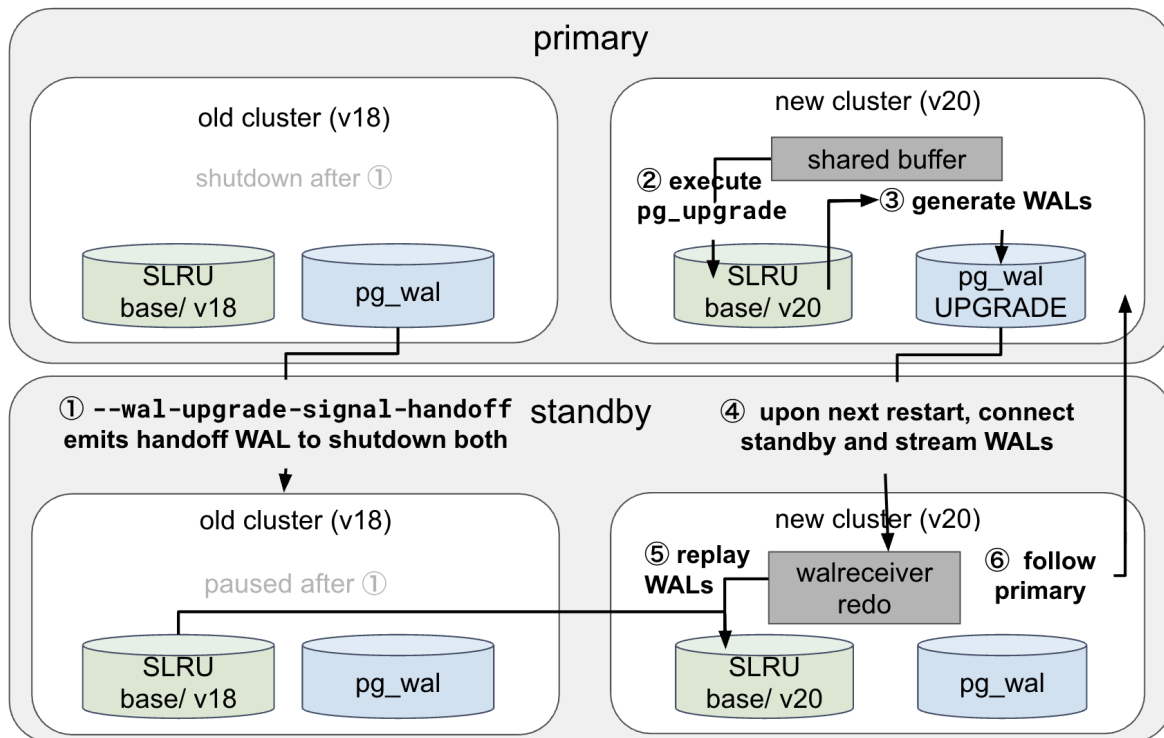
Interface. The feature is exposed through two `pg_upgrade` subcommands. Both share the `--wal-upgrade` prefix to clearly distinguish them from the stock `pg_upgrade` workflow. In a primary-standby deployment, both commands are required to upgrade both primary and standby. In deployments that rely on archived backups, only `--wal-upgrade` is required.

`pg_upgrade --wal-upgrade -d <old> -D <new>` This command is invoked like the stock `pg_upgrade`, using the same old and new data directory arguments and supporting the same transfer modes. It produces a fully upgraded cluster on disk and leaves it shut down. On the next restart, the upgraded cluster starts normally in read-write mode. Unlike the stock workflow, the upgrade is also captured in WAL once upgrade execution completes. The old cluster remains intact, while the upgrade WALs and post-upgrade WALs are streamed to standbys through standard replication.

`pg_upgrade --wal-upgrade-signal-handoff -b <old-bin> -d <old>` This command prepares existing standbys and is executed once on the running primary before the upgrade. The primary emits a handoff WAL record and then shuts down. The handoff record instructs standbys, over their existing replication connections, to pause at the upgrade point so they can transition to the upgraded cluster. During this step, client backends are terminated and the primary is fast-stopped, so no user WAL follows the handoff record.

What if the user wants to roll back? A user may wish to evaluate the upgraded version before fully committing to it, a workflow that stock Postgres also implicitly supports. This is also supported by the proposed interface. There are no behavioral changes in the primary-backup scenario. With standbys, the user can defer `--wal-upgrade-signal-handoff` and run only `--wal-upgrade` on the primary, leaving the standbys unchanged. If the upgrade is accepted, `--wal-upgrade-signal-handoff` can be issued afterward, pausing the old standby and stopping the old primary at that point, with a fresh new cluster taking over from there. The old cluster standby can continue serving reads throughout this period. If the user decides to roll back to the old version, they can simply restart the connection using the old cluster binary and resume the standby from the paused point, if it had been paused.

3. Primary-standby Workflow with the New Interface



① `--wal-upgrade-signal-handoff` writes a handoff record into the live old cluster primary's WAL to signal standby and shuts down the primary. Standbys replay it and pauses.

② `pg_upgrade --wal-upgrade` runs an ordinary in-place upgrade.

③ The upgrade is emitted as a WAL window (CN..COMPLETE): rewritten catalog/system files travel as after-image contents, while the user relations are recorded only by reference by a manifest naming which files to relink. A migrated physical replication slot pinned at CN keeps the window from being recycled. `pg_upgrade` shuts down cleanly.

④ On the operator's next `pg_ctl start`, the primary skips replay and comes up read-write. Operator also restarts standby with a fresh new-version skeleton, pointed at the primary via an ordinary `primary_conninfo` and carrying a `pg_upgrade.signal` sentinel plus `pg_upgrade_standby_old_datadir`. Then, it derives the CN anchor locally from its own retained old data directory and streams the CN..COMPLETE window.

⑤ Startup/redo replays the `UPGRADE_*` WALs to rebuild the catalog and system files from WAL, while the `XLOG_UPGRADE_RELINK` redo links the user relations in `base/` from the standby's retained old `datadir`.

⑥ Once all the upgrade WALs are replayed, it naturally becomes hot standby and follows the primary as a hot standby.

4. How `--wal-upgrade` Enables Continuous Archiving

Assume a deployment consisting of a single primary server, daily base backups, and continuous WAL archiving to separate storage. This scenario is used to illustrate how the stock `pg_upgrade` introduces a durability gap and how the proposed patch eliminates it.

The gap caused during the upgrade process. The upgrade initializes the new cluster with a fresh system identifier and a new WAL history. This breaks WAL/LSN continuity at the upgrade boundary, preventing a base backup taken before the upgrade from being replayed into the upgraded cluster; it can restore only up to the old cluster before upgrade. Until a new base backup of the upgraded cluster is created and archived, no valid backup exists for recovery of the committed transactions processed in the new cluster after upgrade.

How `--wal-upgrade` addresses this.

- ① **Capture.** `pg_upgrade --wal-upgrade` runs the normal upgrade, then captures the after-images of every modification it produced as full-page-image WAL records. These are emitted as replayable WALs on the new cluster's log stream.
- ② **Archive.** All the generated upgrade WALs are archived like any other WAL. The archive therefore holds one continuous replay source spanning the upgrade.
- ③ **Recovery phase 1 (old binary).** On loss of the upgraded cluster, the recovery process can restore the last pre-upgrade base backup and, using `restore_command`, replay archived WAL forward to the upgrade boundary, stopping there without promoting. This recovers every pre-upgrade transaction, on the old version, into the restored data directory.
- ④ **Recovery phase 2 (new-binary phase).** The new-version cluster is started on that same data directory. Detecting `recovery.signal` together with the staged `pg_upgrade.signal` sentinel, it clears the version-compatibility gate and arms upgrade replay. Recovery then continues from the base-backup checkpoint and flows organically through CN and the `UPGRADE_*` window, then keeps applying the archived post-upgrade WAL, recovering the transactions committed after go-live, and promotes.

5. Added TAP Tests

To verify these scenarios, the patch adds two TAP tests. `010_wal_upgrade.pl` validates the durability of the PITR path across the upgrade, along with other scenarios like link mode and

crash recovery, while `011_wal_upgrade_standby.pl` validates whether the upgrade can be propagated by replaying WALs on a standby.

Added TAP test 1: `010_wal_upgrade.pl`

Phase 1: Setup and upgrade.

1. Init an old cluster with `archive_mode=on` and an atomic `archive_command`. Load pre-upgrade data, and take a base backup (`pitr_base`).
2. Run `pg_upgrade --wal-upgrade --initdb` to generate the upgrade WALs. The new cluster inherits `archive_command` and auto-serves.
3. Record the old cluster's final LSN (`old_end_lsn` = the upgrade boundary). From `pg_waldump` of the archive, the segments holding upgrade WALs.
4. Execute post-upgrade transactions, and poll until they are archived.

Phase 2: forcing failure.

5. Stop('immediate') and `rmtree` the new cluster's data directory, only the pre-upgrade base backup and the archive survive.

Phase 3: PITR Phase 1 (old-binary phase).

6. `Init_from_backup` the pre-upgrade base backup into a fresh node.
7. Configure `restore_command`, `recovery_target_lsn` = `old_end_lsn`, `recovery_target_inclusive` = `on`, `recovery_target_action` = `shutdown`, and touch the recovery signal
8. Drive with raw `pg_ctl`. This replays to the boundary and shuts down without promoting.
9. Assert the "shutdown at recovery target" log.

Phase 4: PITR Phase 2 (new-binary phase).

10. On the same restore node, stage the upgrade WAL segments (CN..COMPLETE) from the archive into `pg_wal/` so the new binary's local scan re-anchors recovery at CN.
11. Rewrite the config: strip the `recovery_target_*` lines, keep `restore_command`, set `recovery_target_timeline` = `latest`, and raise the resource GUCs to the primary's values.
12. Touch `recovery.signal` and the empty `pg_upgrade.signal` sentinel, then start the new binary.
13. Assert: once `pg_is_in_recovery()` is false, both pre- and post-upgrade rows are present (count = 800) and `only_on_new` holds 42 — the post-upgrade transactions survived with no new base backup.

Added TAP test 2. `011_wal_upgrade_standby.pl`

Phase 1 — Baseline: old primary + old standby.

1. The operator has an old primary (replication configured, data loaded) and a streaming old-version standby.
2. Verify the standby is a hot standby (`pg_is_in_recovery = t`) holding the pre-upgrade data.

Phase 2 — Signal handoff (reversible pause).

3. The operator runs `pg_upgrade --wal-upgrade-signal-handoff` on the live old primary.
4. It writes `XLOG_UPGRADE_HANDOFF` into the primary's WAL and stops the primary at that point.
5. The old standby replays the record and pauses at the boundary. It stays up, remains in recovery, and keeps serving the pre-upgrade data read-only.

Phase 3 — Upgrade the primary.

6. The operator runs `pg_upgrade --wal-upgrade --initdb`.
7. The new primary auto-serves read-write, data preserved, and a migrated physical retention slot pins the upgrade window so it's streamable.

Phase 4 — Provision a new-version standby.

8. The operator stages a new-version skeleton with `primary_conninfo` to the upgraded primary, `standby.signal`, an empty `pg_upgrade.signal` sentinel, and the retained old datadir named in the `pg_upgrade_standby_old_datadir` GUC (the upgrade WALs contain schema-only, so user relations come from that old datadir).
9. On start the standby derives the CN anchor locally from its old datadir, arms at CN, streams and replays the window, and at `XLOG_UPGRADE_RELINK` places each user relation from the old datadir into the skeleton.
10. The placement mode is the standby's own choice via the GUC configuration option: `pg_upgrade_standby_transfer_mode`. The default mirror reproduces the primary's per-file mode, or the operator can force one (e.g. link). The test covers both.
11. The test asserts the standby converges as a hot standby (`pg_is_in_recovery = t`) matching the primary; that under the default copy/mirror the placed files are independent copies (different inodes from the old datadir, so it stays an intact rollback target) while an explicit link override hardlinks them; that `upgrade_finalized` is set in `pg_control` and the sentinel is left in place; and that a restart comes up as an ordinary hot standby without re-arming at the old CN.

Phase 5 — Abort/rollback.

12. After a handoff, the operator can restart the old primary read-write.
13. The paused standby resumes past the (no-op) handoff and keeps following it, which means that post-upgrade modifications are not preserved.

Phase 6 — Negatives.

14. A streaming skeleton with the `pg_upgrade_standby_old_datadir` GUC unset FATALs at startup rather than silently missing user data.
15. A plain new cluster with no primary and no window comes up as an ordinary empty cluster.

6. Example Use Case: k8s Zalando Spilo

As an example use case, the Zalando Spilo environment is considered. Spilo is the PostgreSQL appliance underlying the Zalando Patroni-based PostgreSQL operator and performs major-version upgrades through its [inplace_upgrade.py](#) procedure. In its unmodified form this procedure upgrades the primary and then reconstructs each streaming replica by transferring the entire upgraded data directory via `rsync`, requiring a complete physical copy for every replica on each upgrade.

Spilo integration workflow. The upgrade procedure is modified to adopt our interface, guarded by an opt-in `USE_WAL_UPGRADE` flag (source available [here](#); see comments labeled 'LEE'). Although this modification has not been evaluated on a live Kubernetes deployment, the objective is to illustrate the operator workflow around `pg_upgrade` rather than Kubernetes-specific behavior. The replica-handling workflow is changed as follows:

1. **Handoff.** Before the primary is shut down, `--wal-upgrade-signal-handoff` is issued against the still-live primary, so the streaming replicas stop deterministically at a well-defined point in the WAL; the same signal shuts the primary down at that point.
2. **Capture.** The upgrade then runs with `--wal-upgrade`, recording the transformation as a replayable WAL window on the primary.
3. **Removal of physical re-imaging.** The original `rsync`-based copy path (`rsyncd` startup, per-replica CHECKPOINT, and directory transfer) is bypassed entirely.
4. **Replay.** Each replica is brought up on the new major version and streams the upgrade window over the existing replication channel. The operator confirms completion by polling each replica until its `database_system_identifier` matches the upgraded cluster.
5. **Note.** The current Spilo prototype drives the handoff and this readiness check, but does not itself stage the new-version replica skeleton / `pg_upgrade.signal` + `old-datadir` configuration that the window-replay path requires. That step is assumed to be handled by the surrounding Patroni bootstrap and has not been validated on a live cluster.